Guia Completa: Deploy Multi-Entorno (Produccion + Staging)

Sistema Blitz — CI/CD con GitHub Actions + Docker Compose + VPS IONOS

Tabla de Contenidos

1. Que vamos a lograr
2. Como funciona todo (la teoria)
3. Que archivos se cambiaron y por que
4. PASO 1: Configurar GitHub Environments
5. PASO 2: Configurar los Secrets en GitHub
6. PASO 3: Preparar el VPS (servidor)
7. PASO 4: Primer deploy manual (verificacion)
8. PASO 5: Probar el flujo automatico
9. Como acceder a cada entorno
10. Diagrama completo del flujo
11. Troubleshooting (cuando algo falla)
12. Referencia rapida de comandos
13. Checklist final

---

1. Que vamos a lograr

Queremos que en **un solo servidor VPS** corran **dos versiones** de la app al mismo tiempo:

```
PRODUCCION (lo que ven los usuarios reales)
   URL:    https://jesuslab135.com
   Rama:   main
   Ruta:   /root/app

STAGING (donde probamos antes de subir a produccion)
   URL:    http://jesuslab135.com:8080
   Rama:   development
   Ruta:   /root/app-dev
```

El flujo automatico es:

```
Tu haces push a "development"
        |
        v
GitHub Actions detecta la rama
        |
```

```
        v
Construye la imagen Docker y la sube a GHCR
        |
        v
Se conecta al VPS por SSH
        |
        v
Descarga la imagen nueva en /root/app-dev
        |
        v
Reinicia los contenedores de staging
        |
        v
Staging actualizado! Prueba en http://jesuslab135.com:8080
```

Lo mismo pasa con `main`, pero despliega en `/root/app` (produccion).

---

2. Como funciona todo (la teoria)

El problema original

Si intentas correr dos copias de la misma app con Docker, todo choca:

Recurso	Produccion quiere	Staging quiere	Resultado
Puerto del backend	8000	8000	ERROR: puerto ocupado
Nombre del contenedor	blitz_backend	blitz_backend	ERROR: nombre duplicado
Base de datos	postgres_data	postgres_data	DESASTRE: comparten datos

La solucion: COMPOSE_PROJECT_NAME

Docker Compose tiene una variable magica llamada `COMPOSE_PROJECT_NAME`. Cuando la defines, Docker le pone ese nombre como **prefijo** a todo:

```
COMPOSE_PROJECT_NAME=blitz-prod
  → Contenedores:  blitz-prod-backend-1, blitz-prod-db-1, ...
  → Volumenes:     blitz-prod_postgres_data
  → Redes:         blitz-prod_blitz-net

COMPOSE_PROJECT_NAME=blitz-dev
  → Contenedores:  blitz-dev-backend-1, blitz-dev-db-1, ...
  → Volumenes:     blitz-dev_postgres_data   ← BASE DE DATOS SEPARADA!
  → Redes:         blitz-dev_blitz-net        ← RED SEPARADA!
```

Eso significa: **un solo archivo `compose.yaml`** sirve para ambos entornos. La diferencia la hace el archivo `.env` que esta en cada carpeta.

Los puertos

Ademas del prefijo, cada entorno usa puertos diferentes:

```
Produccion:                     Staging:
  Nginx:    80 / 443 (HTTPS)     Nginx:    8080 (HTTP)
  Backend:  8000                  Backend:  8001
  PGAdmin:  5050                  PGAdmin:  6060
```

Estos puertos se configuran en el `.env` de cada entorno.

Las imagenes Docker

Las imagenes se etiquetan (tagean) diferente segun la rama:

```
push a main        → ghcr.io/jesuslab135/blitz-backend:latest
push a development → ghcr.io/jesuslab135/blitz-backend:dev
```

El `compose.yaml` usa `${IMAGE_TAG}` para saber cual imagen descargar. Produccion tiene `IMAGE_TAG=latest` en su `.env`, staging tiene `IMAGE_TAG=dev`.

GitHub Environments

GitHub tiene una feature llamada **Environments**. Nos permite guardar secretos diferentes para cada entorno. Ejemplo:

```
Repositorio (secretos compartidos):
  VPS_HOST     = 85.215.x.x
  VPS_USER     = root
  VPS_SSH_KEY  = (tu clave SSH privada)

Environment "production" (secretos solo para prod):
  VPS_DEPLOY_PATH = /root/app

Environment "staging" (secretos solo para staging):
  VPS_DEPLOY_PATH = /root/app-dev
```

El workflow detecta la rama, elige el environment correcto, y automaticamente tiene acceso al `VPS_DEPLOY_PATH` que corresponde.

---

3. Que archivos se cambiaron y por que

Archivos modificados

Archivo	Que se hizo	Por que
`infra/compose.yaml`	Reescrito completo	Eliminamos `container_name:` de todos los servicios, parametrizamos puertos e image tags con variables `${}`, agregamos healthchecks, y usamos `profiles` para certbot y pgadmin
`frontend/Dockerfile` (linea 32)	Fix: `/app/dist` → `/app-dev/dist`	BUG: El WORKDIR es `/app-dev`, pero el COPY del stage 2 buscaba en `/app/dist` (que no existe). El build generaba una imagen vacia
`backend/.github/workflows/deploy.yml`	Reescrito	Ahora detecta la rama (`main` o `development`), tagea la imagen correspondiente, y usa GitHub Environments para deployar al directorio correcto
`frontend/.github/workflows/deploy.yml`	Reescrito	Mismo patron que el backend

Archivos nuevos

Archivo	Que es
`infra/.env.production`	Template del `.env` para produccion. Se copia a `/root/app/.env` en el VPS
`infra/.env.staging`	Template del `.env` para staging. Se copia a `/root/app-dev/.env` en el VPS
`infra/nginx/staging.conf`	Configuracion de Nginx para staging (HTTP simple, sin SSL)
`infra/GUIA-DEPLOY-MULTI-ENTORNO.md`	Esta guia

Bug adicional detectado (no corregido automaticamente)

El `compose.yaml` original tenia `ports: "8001:8001"` para el backend, pero Daphne (el servidor) escucha en el puerto **8000** dentro del contenedor. Eso significa que el mapeo `8001:8001` no conectaba con nada. El compose nuevo lo corrige: `"${BACKEND_PORT:-8000}:8000"` — el puerto externo es configurable, pero el interno siempre es 8000.

---

4. PASO 1: Configurar GitHub Environments

Esto se hace en **ambos repositorios** (backend y frontend) porque cada uno tiene su propio workflow.

4.1 Ir a Settings del repositorio de Backend

1. Abre tu navegador
2. Ve a `https://github.com/jesuslab135/blitz-backend` (o como se llame tu repo)
3. Haz clic en la pestana **Settings** (arriba a la derecha, al lado de "Insights")

```
Code   Issues   Pull requests   Actions   Settings  ← ESTE
```

4. En el menu lateral izquierdo, busca la seccion **"Environments"**

```
Settings
├── General
├── ...
├── Environments  ← ESTE
├── ...
```

4.2 Crear el environment "production"

1. Haz clic en **"New environment"**
2. En "Name" escribe exactamente: `production` (en minusculas, sin espacios)
3. Haz clic en **"Configure environment"**
4. (Opcional pero recomendado) Activa **"Required reviewers"** y agrega tu usuario
   - Esto hace que cada deploy a produccion necesite tu aprobacion manual
   - Es una red de seguridad: si alguien hace push a main por error, el deploy no se ejecuta solo
5. Baja hasta la seccion **"Environment secrets"**
6. Haz clic en **"Add secret"**
7. Agrega este secret:

```
Name:   VPS_DEPLOY_PATH
Value:  /root/app
```

8. Haz clic en **"Add secret"** para guardarlo

4.3 Crear el environment "staging"

1. Vuelve a **Settings > Environments**
2. Haz clic en **"New environment"**
3. En "Name" escribe exactamente: `staging`
4. Haz clic en **"Configure environment"**
5. NO actives "Required reviewers" (staging se deploya automaticamente)
6. Baja a **"Environment secrets"**
7. Agrega:

```
Name:   VPS_DEPLOY_PATH
Value:  /root/app-dev
```

8. Guarda

4.4 Repetir para el repositorio de Frontend

Haz **exactamente lo mismo** en el repo del frontend (`blitz-frontend` o como se llame):

1. Settings > Environments > New environment > `production` > secret `VPS_DEPLOY_PATH` = `/root/app`
2. Settings > Environments > New environment > `staging` > secret `VPS_DEPLOY_PATH` = `/root/app-dev`

Verificacion

Al terminar, ambos repos deben tener esto:

```
Backend repo:
  Environments:
    production  → VPS_DEPLOY_PATH = /root/app
    staging     → VPS_DEPLOY_PATH = /root/app-dev

Frontend repo:
  Environments:
    production  → VPS_DEPLOY_PATH = /root/app
    staging     → VPS_DEPLOY_PATH = /root/app-dev
```

---

5. PASO 2: Configurar los Secrets en GitHub

Los secrets son valores sensibles (contrasenas, claves SSH) que GitHub guarda encriptados y los inyecta en los workflows.

5.1 Secrets a nivel de REPOSITORIO (compartidos entre environments)

Estos secretos son los mismos para produccion y staging porque usamos el **mismo servidor VPS**.

1. En tu repo de Backend, ve a: **Settings > Secrets and variables > Actions**

```
Settings
├── ...
├── Secrets and variables
│   ├── Actions  ← ESTE
│   ├── ...
```

2. Haz clic en **"New repository secret"**
3. Agrega estos 3 secretos, uno por uno:

Secret 1: VPS_HOST

```
Name:   VPS_HOST
Value:  85.215.xxx.xxx    ← Pon la IP publica de tu VPS IONOS
```

> Para encontrar tu IP: entra a tu panel de IONOS > Servidores > tu VPS > la IP esta en la info general.

Secret 2: VPS_USER

```
Name:   VPS_USER
Value:  root
```

> Si usas otro usuario SSH (como `deploy`), pon ese.

Secret 3: VPS_SSH_KEY

```
Name:   VPS_SSH_KEY
Value:  (contenido completo de tu clave privada SSH)
```

Como obtener la clave SSH:

Si ya te conectas al VPS desde tu computadora con `ssh root@85.215.xxx.xxx`, tu clave privada esta en:

```
# En tu computadora LOCAL (no en el VPS)
cat ~/.ssh/id_rsa
# O si usas ed25519:
cat ~/.ssh/id_ed25519
```

Copia TODO el contenido, incluyendo las lineas `-----BEGIN` y `-----END`:

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUA...
(muchas lineas de texto)
...
-----END OPENSSH PRIVATE KEY-----
```

Pega eso completo como valor del secret `VPS_SSH_KEY`.

Si NO tienes clave SSH configurada:

```
# En tu computadora LOCAL
ssh-keygen -t ed25519 -C "github-actions-deploy"
# Te pregunta donde guardar (Enter para default)
```

```
# Te pregunta passphrase (Enter para dejar vacia — necesario para CI/CD)

# Copiar la clave PUBLICA al VPS
ssh-copy-id root@85.215.xxx.xxx
# Te pedira la contrasena del VPS una ultima vez

# Verificar que funciona (no deberia pedir contrasena)
ssh root@85.215.xxx.xxx "echo Conexion exitosa"

# Copiar la clave PRIVADA para GitHub
cat ~/.ssh/id_ed25519
# Pega este contenido como VPS_SSH_KEY en GitHub
```

5.2 Repetir en el repo de Frontend

Agrega los mismos 3 secretos en el repo del frontend:

- `VPS_HOST` (misma IP)
- `VPS_USER` (mismo usuario)
- `VPS_SSH_KEY` (misma clave)

Verificacion

Al terminar, cada repo debe tener:

```
Repository secrets (nivel repo):
  VPS_HOST      = 85.215.xxx.xxx
  VPS_USER      = root
  VPS_SSH_KEY   = -----BEGIN OPENSSH PRIVATE KEY-----...

Environment "production":
  VPS_DEPLOY_PATH = /root/app

Environment "staging":
  VPS_DEPLOY_PATH = /root/app-dev
```

> NOTA: Si antes tenias un secret `VPS_DEPLOY_PATH` a nivel de repositorio (no de environment), **eliminalo**. Los secrets de environment tienen prioridad, pero es mejor no tener duplicados para evitar confusion.

---

6. PASO 3: Preparar el VPS (servidor)

Conectate a tu VPS por SSH:

```
ssh root@85.215.xxx.xxx
```

6.1 Verificar que Docker esta instalado

```
docker --version
# Debe mostrar algo como: Docker version 24.x.x o superior

docker compose version
# Debe mostrar algo como: Docker Compose version v2.x.x
```

Si no tienes Docker instalado:

```
curl -fsSL https://get.docker.com | sh
```

6.2 Crear la estructura de directorios

```
# Crear directorio de STAGING (produccion ya existe en /root/app)
mkdir -p /root/app-dev/nginx
```

Si `/root/app` no existe todavia:

```
mkdir -p /root/app/nginx
```

6.3 Verificar la estructura

```
ls -la /root/app/
# Debe existir (produccion actual)

ls -la /root/app-dev/
# Debe existir (lo acabamos de crear)
```

6.4 Subir el compose.yaml a ambos directorios

Desde tu **computadora local**, sube el archivo con `scp`:

```
# Desde la carpeta infra/ de tu proyecto local
scp compose.yaml root@85.215.xxx.xxx:/root/app/compose.yaml
scp compose.yaml root@85.215.xxx.xxx:/root/app-dev/compose.yaml
```

> **Alternativa**: Si prefieres hacerlo desde el VPS directamente:

```
# En el VPS
nano /root/app/compose.yaml
# Pega el contenido del compose.yaml
# Ctrl+O para guardar, Ctrl+X para salir

# Copiar al otro directorio
cp /root/app/compose.yaml /root/app-dev/compose.yaml
```

6.5 Subir las configuraciones de Nginx

```
# Desde tu computadora local, carpeta infra/

# Nginx para PRODUCCION (con SSL)
scp nginx/default.conf root@85.215.xxx.xxx:/root/app/nginx/default.conf

# Nginx para STAGING (sin SSL, HTTP simple)
scp nginx/staging.conf root@85.215.xxx.xxx:/root/app-dev/nginx/default.conf
```

> **Importante**: El archivo se llama `staging.conf` en tu repo, pero se copia como `default.conf` en el VPS. Esto es porque el `compose.yaml` siempre monta `./nginx/default.conf`.

6.6 Crear el archivo .env de PRODUCCION

En el VPS:

```
nano /root/app/.env
```

Pega esto y **edita cada valor** con tus datos reales:

```
# --- Docker Compose ---
COMPOSE_PROJECT_NAME=blitz-prod
COMPOSE_PROFILES=ssl
IMAGE_TAG=latest

# --- Puertos (Produccion) ---
HTTP_PORT=80
HTTPS_PORT=443
BACKEND_PORT=8000
PGADMIN_PORT=5050

# --- Base de Datos ---
DB_USER=tu_usuario_de_bd_produccion
DB_PASSWORD=tu_password_seguro_produccion
DB_NAME=blitz_production

# --- Django ---
```

```
SECRET_KEY=tu-secret-key-de-produccion-algo-largo-y-aleatorio

# --- Firebase ---
FIREBASE_CREDENTIALS_PATH=/app-dev/core/squapup-3ab7a-firebase-adminsdk-fbsvc-
53c50f45a0.json

# --- Stripe (LIVE keys para produccion) ---
STRIPE_SECRET_KEY=sk_live_xxx
STRIPE_PUBLISHABLE_KEY=pk_live_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx

# --- PGAdmin ---
PGADMIN_EMAIL=admin@blitz.com
PGADMIN_PASSWORD=password_seguro_pgadmin
```

Guarda con `Ctrl+O`, `Enter`, `Ctrl+X`.

6.7 Crear el archivo .env de STAGING

```
nano /root/app-dev/.env
```

Pega esto y **edita cada valor**:

```
# --- Docker Compose ---
COMPOSE_PROJECT_NAME=blitz-dev
IMAGE_TAG=dev

# --- Puertos (Staging — DIFERENTES de produccion!) ---
HTTP_PORT=8080
HTTPS_PORT=8443
BACKEND_PORT=8001
PGADMIN_PORT=6060

# --- Base de Datos (DIFERENTE de produccion!) ---
DB_USER=blitz_dev_user
DB_PASSWORD=dev_password_seguro
DB_NAME=blitz_development

# --- Django ---
SECRET_KEY=otra-secret-key-diferente-para-staging

# --- Firebase (pueden ser las mismas credenciales) ---
FIREBASE_CREDENTIALS_PATH=/app-dev/core/squapup-3ab7a-firebase-adminsdk-fbsvc-
53c50f45a0.json

# --- Stripe (TEST keys — nunca uses live keys en staging!) ---
STRIPE_SECRET_KEY=sk_test_xxx
STRIPE_PUBLISHABLE_KEY=pk_test_xxx
STRIPE_WEBHOOK_SECRET=whsec_test_xxx
```

```
# --- PGAdmin ---
PGADMIN_EMAIL=admin@blitz.local
PGADMIN_PASSWORD=admin
```

Guarda con `Ctrl+O`, `Enter`, `Ctrl+X`.

6.8 Verificar que todo esta en su lugar

```
echo "=== PRODUCCION ==="
ls -la /root/app/
# Debe mostrar: compose.yaml, .env, nginx/

echo "=== STAGING ==="
ls -la /root/app-dev/
# Debe mostrar: compose.yaml, .env, nginx/

echo "=== NGINX PRODUCCION ==="
ls -la /root/app/nginx/
# Debe mostrar: default.conf

echo "=== NGINX STAGING ==="
ls -la /root/app-dev/nginx/
# Debe mostrar: default.conf

echo "=== VERIFICAR .env PRODUCCION ==="
cat /root/app/.env | head -5
# Debe mostrar: COMPOSE_PROJECT_NAME=blitz-prod

echo "=== VERIFICAR .env STAGING ==="
cat /root/app-dev/.env | head -5
# Debe mostrar: COMPOSE_PROJECT_NAME=blitz-dev
```

---

7. PASO 4: Primer deploy manual (verificacion)

Antes de confiar en el CI/CD automatico, vamos a levantar todo manualmente para verificar que funciona.

7.1 Asegurarse de que las imagenes existen en GHCR

Las imagenes Docker deben existir en GitHub Container Registry. Si nunca has hecho push a `development`, la imagen `:dev` no existira todavia.

Opcion A: Si ya tienes imagenes publicadas:

```
# En el VPS, verificar
docker pull ghcr.io/jesuslab135/blitz-backend:latest
docker pull ghcr.io/jesuslab135/blitz-frontend:latest
```

```
docker pull ghcr.io/jesuslab135/blitz-backend:dev
docker pull ghcr.io/jesuslab135/blitz-frontend:dev
```

Opcion B: Si la imagen `:dev` no existe, primero haz un push a `development` para que el workflow la construya (ver Paso 5).

7.2 Levantar PRODUCCION

```
cd /root/app

# Verificar que Docker Compose lee el .env correctamente
docker compose config | head -20
# Debe mostrar "name: blitz-prod" y las variables resueltas

# Levantar todos los servicios
docker compose up -d

# Ver el estado
docker compose ps
```

Deberias ver algo asi:

```
NAME                      IMAGE                                          STATUS
blitz-prod-backend-1      ghcr.io/jesuslab135/blitz-backend:latest      Up (healthy)
blitz-prod-frontend-1     ghcr.io/jesuslab135/blitz-frontend:latest     Up
blitz-prod-db-1           postgres:16                                    Up
(healthy)
blitz-prod-redis-1        redis:7-alpine                                 Up
(healthy)
blitz-prod-nginx-1        nginx:alpine                                   Up
blitz-prod-certbot-1      certbot/certbot                                Up
```

> **Fijate en los nombres**: todos empiezan con `blitz-prod-`. Eso confirma que el `COMPOSE_PROJECT_NAME` funciona.

7.3 Levantar STAGING

```
cd /root/app-dev

# Verificar configuracion
docker compose config | head -20
# Debe mostrar "name: blitz-dev"

# Levantar
docker compose up -d
```

```
# Ver estado
docker compose ps
```

Deberias ver:

```
NAME                    IMAGE                                          STATUS
blitz-dev-backend-1     ghcr.io/jesuslab135/blitz-backend:dev         Up (healthy)
blitz-dev-frontend-1    ghcr.io/jesuslab135/blitz-frontend:dev        Up
blitz-dev-db-1          postgres:16                                    Up (healthy)
blitz-dev-redis-1       redis:7-alpine                                 Up (healthy)
blitz-dev-nginx-1       nginx:alpine                                   Up
```

> Nota: certbot NO aparece en staging porque no tiene `COMPOSE_PROFILES=ssl`.

7.4 Verificar que no hay conflictos de puertos

```
# Ver TODOS los contenedores corriendo (ambos entornos)
docker ps --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"
```

Debe mostrar algo como:

```
NAMES                    PORTS                                          STATUS
blitz-prod-nginx-1       0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp Up
blitz-prod-backend-1     0.0.0.0:8000->8000/tcp                     Up
blitz-dev-nginx-1        0.0.0.0:8080->80/tcp, 0.0.0.0:8443->443  Up
blitz-dev-backend-1      0.0.0.0:8001->8000/tcp                     Up
```

Puntos clave:

- Produccion usa puertos **80, 443, 8000**
- Staging usa puertos **8080, 8443, 8001**
- NO hay conflictos

7.5 Verificar que los volumenes estan separados

```
docker volume ls | grep blitz
```

Debe mostrar:

```
blitz-prod_postgres_data
blitz-prod_redis_data
blitz-prod_frontend_dist
```

```
blitz-prod_certbot_conf
blitz-prod_certbot_www
blitz-dev_postgres_data      ← BD SEPARADA!
blitz-dev_redis_data
blitz-dev_frontend_dist
```

`blitz-prod_postgres_data` y `blitz-dev_postgres_data` son volumenes DIFERENTES. La base de datos de staging nunca tocara la de produccion.

7.6 Probar acceso web

```
# Produccion (HTTPS)
curl -s -o /dev/null -w "HTTP %{http_code}\n" https://jesuslab135.com

# Staging (HTTP en puerto 8080)
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://jesuslab135.com:8080

# Backend produccion
curl -s -o /dev/null -w "HTTP %{http_code}\n" https://jesuslab135.com/api/

# Backend staging
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://jesuslab135.com:8080/api/
```

Si todo da `HTTP 200` (o `301` para el redirect HTTP→HTTPS en prod), funciona.

---

8. PASO 5: Probar el flujo automatico

8.1 Probar deploy a Staging

En tu **computadora local**:

```
# Ir al repo del backend
cd backend

# Asegurarte de estar en development
git checkout development

# Hacer un cambio pequeno (por ejemplo, agregar un comentario)
echo "# test staging deploy" >> README.md

# Commit y push
git add README.md
git commit -m "test: verificar deploy automatico a staging"
git push origin development
```

8.2 Ver el workflow en accion

1. Ve a tu repo en GitHub
2. Haz clic en la pestana **"Actions"**
3. Deberias ver el workflow **"Build & Deploy Backend"** ejecutandose
4. Haz clic en el para ver el detalle

Veras 3 jobs:

```
setup        ☑ (detecta: environment=staging, image_tag=dev)
   |
   v
build-and-push  🔄 (construyendo imagen...)
   |
   v
deploy          ⏳ (esperando build...)
```

5. Cuando termine, el job `deploy` mostrara "staging" como environment
6. En los logs del deploy veras:

```
=== Desplegando staging ===
=== Pulling imagen: ghcr.io/jesuslab135/blitz-backend:dev ===
...
=== Deploy completado: staging / abc123 ===
```

8.3 Verificar en el VPS

```
ssh root@85.215.xxx.xxx
cd /root/app-dev
docker compose ps
# El backend debe estar corriendo con la imagen nueva

docker compose logs backend --tail 20
# Debe mostrar que arranco correctamente
```

8.4 Probar deploy a Produccion

```
# En tu computadora local
cd backend
git checkout main
git merge development  # O hacer un PR y merge
git push origin main
```

Ve a Actions de nuevo. Esta vez:

- Si activaste "Required reviewers" en el environment de produccion, GitHub te pedira aprobacion antes del job `deploy`
- Ve a Actions > el workflow > haz clic en "Review deployments" > "Approve"
- El deploy procedera

8.5 Repetir con el Frontend

Haz lo mismo con el repo del frontend:

```
cd frontend
git checkout development
# Hacer un cambio
git add .
git commit -m "test: verificar deploy frontend staging"
git push origin development
```

---

9. Como acceder a cada entorno

Produccion

Servicio	URL
Frontend (web)	`https://jesuslab135.com`
Backend API	`https://jesuslab135.com/api/`
WebSocket	`wss://jesuslab135.com/ws/`
Swagger docs	`https://jesuslab135.com/api/schema/swagger-ui/`

Staging

Servicio	URL
Frontend (web)	`http://jesuslab135.com:8080`
Backend API	`http://jesuslab135.com:8080/api/`
WebSocket	`ws://jesuslab135.com:8080/ws/`
Swagger docs	`http://jesuslab135.com:8080/api/schema/swagger-ui/`
Backend directo (sin nginx)	`http://jesuslab135.com:8001`

PGAdmin (solo si se activa)

```
# Activar PGAdmin en staging
cd /root/app-dev
docker compose --profile tools up -d pgadmin
# Acceder en http://jesuslab135.com:6060
```

> Nota: Staging no tiene SSL. Para un entorno de pruebas esto esta bien. Si necesitas SSL para staging en el futuro, puedes configurar un subdominio `dev.jesuslab135.com` con su propio certificado.

---

10. Diagrama completo del flujo

```
DEVELOPER (tu computadora)
│
├── git push development ──────────────────────────────┐
│                                                      │
├── git push main ──────────────────────────┐          │
│                                           │          │
│                                           ▼          ▼
│                                   ┌─ GITHUB ACTIONS ─┐
│                                   │                  │
│                                   │  1. setup job    │
│                                   │     detecta rama │
│                                   │                  │
│                                   │  main → production│
│                                   │  dev  → staging  │
│                                   │                  │
│                                   │  2. build-and-push│
│                                   │     construye img │
│                                   │     sube a GHCR  │
│                                   │                  │
│                                   │  main → :latest  │
│                                   │  dev  → :dev     │
│                                   │                  │
│                                   │  3. deploy job   │
│                                   │     SSH al VPS   │
│                                   └──────────────────┘
│                                           │
│                                   ┌───────┴──────────┐
│                                   │                  │
│                               ┌───▼──────┐       ┌───▼──────┐
│                               │  VPS     │       │  VPS     │
│                               │ /root/app │       │ /root/app-dev│
│                               │          │       │          │
│                               │ .env:    │       │ .env:    │
│                               │ blitz-prod│       │ blitz-dev │
│                               │ :latest  │       │ :dev     │
│                               │ port 80  │       │ port 8080 │
│                               └──────────┘       └──────────┘
│                                    │                  │
│                                    ▼                  ▼
│                           ┌──────────────┐     ┌──────────────┐
│                           │ PRODUCCION   │     │ STAGING      │
│                           │              │     │              │
│                           │ nginx :80/443 │     │ nginx :8080  │
│                           │ backend :8000 │     │ backend :8001 │
```

```
│                           │  db (separada) │        │  db (separada) │
│                           │  redis         │        │  redis         │
│                           │  certbot (SSL) │        │  (sin SSL)     │
│                           └────────────────┘        └────────────────┘
│
│  Acceso:
│  https://jesuslab135.com         http://jesuslab135.com:8080
```

---

11. Troubleshooting (cuando algo falla)

"Error: port is already allocated"

Causa: Otro servicio ya usa ese puerto.

```
# Ver que usa el puerto 8080
ss -tlnp | grep 8080

# Si es un contenedor viejo, paralo
docker ps -a | grep 8080
docker stop <container_id>
docker rm <container_id>

# Re-levantar
cd /root/app-dev && docker compose up -d
```

"Error: image not found" o "manifest unknown"

Causa: La imagen `:dev` no ha sido construida todavia.

```
# Verificar que imagenes existen
docker images | grep blitz

# Solucion: hacer un push a development para triggear el build
# O ejecutar el workflow manualmente desde GitHub Actions
```

Para ejecutar manualmente:

1. Ve a GitHub > tu repo > Actions
2. Haz clic en el workflow "Build & Deploy Backend"
3. Haz clic en "Run workflow"
4. Selecciona la rama `development`
5. Haz clic en "Run workflow"

"Permission denied" en el deploy SSH

Causa: La clave SSH no coincide.

```
# En tu computadora, verificar que puedes conectarte
ssh -i ~/.ssh/id_ed25519 root@85.215.xxx.xxx "echo OK"

# Si falla, regenerar y resubir la clave
ssh-keygen -t ed25519 -C "github-actions"
ssh-copy-id -i ~/.ssh/id_ed25519.pub root@85.215.xxx.xxx

# Actualizar el secret VPS_SSH_KEY en GitHub con la nueva clave privada
cat ~/.ssh/id_ed25519
```

Los contenedores inician pero la app no funciona

```
# Ver logs del backend
cd /root/app-dev
docker compose logs backend --tail 50

# Errores comunes:
# "could not connect to server" → db no esta lista, esperar unos segundos
# "FIREBASE_CREDENTIALS_PATH" → ruta incorrecta en .env
# "No module named..." → imagen desactualizada, hacer pull

# Reiniciar todo
docker compose down
docker compose up -d
docker compose logs -f
```

"db is unhealthy" o "depends_on condition failed"

```
# Ver logs de la base de datos
cd /root/app-dev
docker compose logs db --tail 30

# Error comun: password authentication failed
# → Verificar DB_USER y DB_PASSWORD en .env
# → Si cambiaste credenciales, necesitas resetear el volumen:
docker compose down
docker volume rm blitz-dev_postgres_data  # CUIDADO: borra datos de staging
docker compose up -d
```

Staging funciona pero Produccion no (o viceversa)

```
# Comparar las configuraciones
diff <(cd /root/app && docker compose config) <(cd /root/app-dev && docker compose
config)
```

```
# Verificar que los .env son diferentes
head -3 /root/app/.env
# COMPOSE_PROJECT_NAME=blitz-prod

head -3 /root/app-dev/.env
# COMPOSE_PROJECT_NAME=blitz-dev
```

El workflow de GitHub Actions falla en el job "deploy"

1. Ve a Actions > el workflow fallido > job "deploy"
2. Expande el step "Deploy to VPS"
3. Lee el error

Errores comunes:

- `ssh: connect to host: Connection refused` → El VPS tiene el firewall cerrado para SSH (puerto 22)
- `docker compose: command not found` → Docker Compose V2 no esta instalado en el VPS
- `no configuration file provided: not found` → No existe `compose.yaml` en el directorio de deploy

Nginx no arranca con el error "ssl_certificate not found"

Esto pasa en produccion si los certificados SSL no existen todavia.

```
cd /root/app

# Solucion 1: Si es la primera vez, ejecutar el script init-letsencrypt.sh
# (ver documentacion existente en infra/init-letsencrypt.sh)

# Solucion 2: Temporalmente usar la config HTTP-only
cp /root/app-dev/nginx/default.conf /root/app/nginx/default.conf
docker compose restart nginx
# Luego configurar SSL
```

---

12. Referencia rapida de comandos

En el VPS

```
# ==========================================
# PRODUCCION (/root/app)
# ==========================================

cd /root/app

# Ver estado de todos los contenedores
docker compose ps
```

```
# Ver logs del backend (ultimas 50 lineas)
docker compose logs backend --tail 50

# Ver logs en tiempo real (Ctrl+C para salir)
docker compose logs -f

# Reiniciar un servicio especifico
docker compose restart backend

# Reiniciar todo
docker compose down && docker compose up -d

# Actualizar imagen manualmente (sin CI/CD)
docker compose pull backend
docker compose up -d backend

# Entrar al shell de Django
docker compose exec backend python3 manage.py shell

# Ejecutar migraciones
docker compose exec backend python3 manage.py migrate

# Ver la configuracion resuelta (debug)
docker compose config

# ==========================================
# STAGING (/root/app-dev)
# ==========================================

cd /root/app-dev

# Exactamente los mismos comandos, pero desde este directorio
docker compose ps
docker compose logs backend --tail 50
docker compose restart backend
# etc...

# ==========================================
# COMANDOS GLOBALES (ambos entornos)
# ==========================================

# Ver TODOS los contenedores del VPS
docker ps --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"

# Ver todos los volumenes
docker volume ls | grep blitz

# Ver uso de disco
docker system df

# Limpiar imagenes no usadas
docker image prune -f
```

```
# Limpiar TODO lo no usado (imagenes, containers, networks)
docker system prune -f
# CUIDADO: no usar en produccion sin pensar

# Activar PGAdmin en staging
cd /root/app-dev && docker compose --profile tools up -d pgadmin

# Desactivar PGAdmin
cd /root/app-dev && docker compose --profile tools stop pgadmin
```

En GitHub (manualmente)

```
Ejecutar workflow manualmente:
  Repo > Actions > "Build & Deploy Backend" > "Run workflow"
  > Seleccionar rama (main o development) > "Run workflow"

Ver logs de un deploy:
  Repo > Actions > clic en el workflow > clic en el job "deploy"

Re-ejecutar un workflow fallido:
  Repo > Actions > clic en el workflow fallido > "Re-run all jobs"
```

---

13. Checklist final

Usa esta lista para verificar que todo esta configurado correctamente.

GitHub (hacer en AMBOS repos: backend y frontend)

- ☐ Secret de repositorio: `VPS_HOST` creado con la IP del VPS
- ☐ Secret de repositorio: `VPS_USER` creado (normalmente `root`)
- ☐ Secret de repositorio: `VPS_SSH_KEY` creado con la clave privada SSH completa
- ☐ Environment `production` creado
- ☐ Environment `production`: secret `VPS_DEPLOY_PATH` = `/root/app`
- ☐ (Opcional) Environment `production`: "Required reviewers" activado
- ☐ Environment `staging` creado
- ☐ Environment `staging`: secret `VPS_DEPLOY_PATH` = `/root/app-dev`
- ☐ Verificar que NO existe un secret `VPS_DEPLOY_PATH` a nivel de repositorio (solo en environments)

Archivos en el repositorio (commitear y pushear)

- ☐ `infra/compose.yaml` — version parametrizada (sin `container_name:`, con `${}` variables)
- ☐ `backend/.github/workflows/deploy.yml` — version multi-entorno
- ☐ `frontend/.github/workflows/deploy.yml` — version multi-entorno
- ☐ `frontend/Dockerfile` — fix en linea 32 (`/app-dev/dist` en vez de `/app/dist`)

VPS — Directorio de Produccion (/root/app)

- [ ] `/root/app/compose.yaml` existe (mismo archivo que infra/compose.yaml)
- [ ] `/root/app/.env` existe con `COMPOSE_PROJECT_NAME=blitz-prod`
- [ ] `/root/app/.env` tiene `COMPOSE_PROFILES=ssl`
- [ ] `/root/app/.env` tiene `IMAGE_TAG=latest`
- [ ] `/root/app/.env` tiene `BACKEND_PORT=8000`
- [ ] `/root/app/.env` tiene `HTTP_PORT=80` y `HTTPS_PORT=443`
- [ ] `/root/app/.env` tiene credenciales de BD de produccion
- [ ] `/root/app/.env` tiene Stripe LIVE keys (o test si aun no estas en produccion real)
- [ ] `/root/app/nginx/default.conf` existe (version con SSL)
- [ ] Certificados SSL configurados (Let's Encrypt)

VPS — Directorio de Staging (/root/app-dev)

- [ ] `/root/app-dev/compose.yaml` existe (mismo archivo)
- [ ] `/root/app-dev/.env` existe con `COMPOSE_PROJECT_NAME=blitz-dev`
- [ ] `/root/app-dev/.env` NO tiene `COMPOSE_PROFILES=ssl`
- [ ] `/root/app-dev/.env` tiene `IMAGE_TAG=dev`
- [ ] `/root/app-dev/.env` tiene `BACKEND_PORT=8001`
- [ ] `/root/app-dev/.env` tiene `HTTP_PORT=8080`
- [ ] `/root/app-dev/.env` tiene credenciales de BD DIFERENTES a produccion
- [ ] `/root/app-dev/.env` tiene Stripe TEST keys
- [ ] `/root/app-dev/nginx/default.conf` existe (version staging sin SSL)

Verificacion final

- [ ] `docker compose ps` en `/root/app` muestra contenedores con prefijo `blitz-prod-`
- [ ] `docker compose ps` en `/root/app-dev` muestra contenedores con prefijo `blitz-dev-`
- [ ] `docker volume ls | grep blitz` muestra volumenes separados (`blitz-prod_postgres_data` y `blitz-dev_postgres_data`)
- [ ] `https://jesuslab135.com` carga (produccion)
- [ ] `http://jesuslab135.com:8080` carga (staging)
- [ ] Push a `development` triggerea deploy a staging
- [ ] Push a `main` triggerea deploy a produccion

Apendice: Tabla de puertos

Puerto	Servicio	Entorno	Notas
80	Nginx HTTP	Produccion	Redirige a 443 (HTTPS)
443	Nginx HTTPS	Produccion	SSL con Let's Encrypt
8000	Backend directo	Produccion	Accesible via nginx en /api/
5050	PGAdmin	Produccion	Solo si se activa profile "tools"
8080	Nginx HTTP	Staging	Punto de entrada principal staging
8443	(reservado)	Staging	Para futuro SSL en staging

Puerto	Servicio	Entorno	Notas
8001	Backend directo	Staging	Accesible via nginx en /api/
6060	PGAdmin	Staging	Solo si se activa profile "tools"

Puertos internos (no expuestos al exterior)

Puerto	Servicio	Notas
5432	PostgreSQL	Solo accesible dentro de la red Docker
6379	Redis	Solo accesible dentro de la red Docker

Apendice: Estructura final del VPS

```
/root/
├── app/                          ← PRODUCCION
│   ├── compose.yaml              ← Mismo archivo en ambos
│   ├── .env                      ← blitz-prod, :latest, port 80
│   └── nginx/
│       └── default.conf          ← Con SSL (Let's Encrypt)
│
└── app-dev/                      ← STAGING
    ├── compose.yaml              ← Mismo archivo en ambos
    ├── .env                      ← blitz-dev, :dev, port 8080
    └── nginx/
        └── default.conf          ← Sin SSL (HTTP simple)
```

```
Docker resources en el servidor:

Contenedores:
  blitz-prod-nginx-1          blitz-dev-nginx-1
  blitz-prod-backend-1        blitz-dev-backend-1
  blitz-prod-frontend-1       blitz-dev-frontend-1
  blitz-prod-db-1             blitz-dev-db-1
  blitz-prod-redis-1          blitz-dev-redis-1
  blitz-prod-certbot-1        (no certbot en staging)

Volumenes (datos separados):
  blitz-prod_postgres_data    blitz-dev_postgres_data
  blitz-prod_redis_data       blitz-dev_redis_data
  blitz-prod_frontend_dist    blitz-dev_frontend_dist
  blitz-prod_certbot_conf     blitz-dev_certbot_conf
  blitz-prod_certbot_www      blitz-dev_certbot_www

Redes (aisladas):
  blitz-prod_blitz-net        blitz-dev_blitz-net
```